



**Embedded System and Design Internship
Project Report
Home Automation System**

Name: Aditi Srivastava

University: Vellore Institute of Technology, Chennai

Registration Number: 21BEC1214

Submitted on: 11 July 2024

INDEX

Topic	Page No.
1. Introduction	3
2. Project Specification	4-5
2.1 Function Requirements	
2.2 Non-Function Requirements	
3. Hardware Architecture	6-8
3.1 List of Hardware Components Required	
4. Software Architecture	9-17
5. Code	18-22
6. Testing and Result	13-26
7. Future work	27-28
8. Conclusion	29

Introduction

In this fast-paced world, convenience and efficiency have become paramount in our daily lives. **Home automation systems** serve as a bridge to smart living by integrating various household devices, allowing for seamless control and monitoring. This project aims to create a home automation system that enhances the comfort, safety, and energy efficiency of homes.

The need for a home automation system arises from the desire to enhance comfort, convenience, energy efficiency, and safety in modern living environments. By automating routine tasks such as controlling lights and fans, the system offers a higher level of comfort, allowing residents to enjoy a tailored living environment without the need for constant manual adjustments. This is particularly beneficial for the elderly and individuals with disabilities, providing them with greater independence and ease of living. Additionally, the system promotes energy efficiency by ensuring that appliances are only in use when necessary, thereby reducing energy consumption and lowering electricity bills. This not only has economic benefits for homeowners but also contributes to environmental sustainability by conserving resources and reducing carbon footprints. Safety and security are further enhanced through automation, as automated lighting can deter potential intruders by creating the illusion of occupancy even when no one is home. Overall, a home automation system addresses the increasing demand for smart living solutions, offering significant improvements in comfort, energy efficiency, and safety, which collectively contribute to a better quality of life and support broader societal efforts towards sustainability and technological advancement.

During the internship at Maven Silicon, this project is developed that uses embedded systems to create a smart home environment. The system automates lighting and temperature control using sensors, relays, and IoT components, adjusting based on the conditions in the room and occupancy. This project showcases the practical skills and knowledge gained during internship, emphasizing how embedded systems can be applied to improve everyday life with advanced technology.

Project Specification

Functional Requirements

1. Automatic Light Control:

- Detection of Presence: The system must detect when a person enters or exits the room using an ultrasonic sensor.
- Daylight Detection: The system must determine the presence or absence of daylight in the room using an LDR sensor.
- Light Activation: When a person enters the room and there is no daylight, the system should automatically switch on the bulb.
- Light Deactivation: When the person exits the room, the system should automatically switch off the bulb.

2. Automatic Fan Control:

- Temperature Monitoring: The system must continuously monitor the room temperature using a DHT11 sensor.
- Fan Activation: If the room temperature exceeds a predefined threshold, the system should automatically switch on the fan.
- Fan Deactivation: If the room temperature drops below the threshold, the system should automatically switch off the fan.

3. Manual Mode:

- Remote Control: The system must allow the user to manually control the fan and bulb through a mobile phone connected to the same home network. This control should be accessible by browsing to the appropriate IP address of the home automation system.
- Mode Restriction: The system should restrict manual control of the fan and bulb when it is in automatic mode. The manual mode should be clearly distinguishable from the automatic mode, and the user should be able to switch between modes as needed.

4. Network Connectivity:

- WiFi Connection: The system must connect to the home WiFi network to enable remote control functionality.

5. Home Appliances specification:

- This system is works for two home appliances, a light bulb and a fan, both connected to a 230V power supply with the help of relays. This configuration allows for both automated and manual control one at a time.

These functional requirements ensure that the home automation system operates efficiently and provides the desired levels of comfort, convenience, and energy efficiency.

Non-Functional Requirements

1. **Automated activation of Bulb and Fan:** The light bulb and fan should turn on within one minute of detecting a person's presence in the room and according to specified condition of functional requirements. This swift response is crucial for providing immediate illumination, thereby enhancing convenience and safety.
2. **Manual activation of Bulb and Fan:** The light bulb and fan should turn on within one minute of pressing button in mobile. This swift response is crucial for providing immediate illumination, thereby enhancing convenience and safety and minimizing the delay.
3. **Restriction:** Between automated and manual mode only one should work at a time. User cannot access other if one is in function. And system should work for 0 to 60 degree Celsius for AC supply of 240V.

Hardware Architecture

In this project of home automation system, I have selected specific sensors and components to achieve the desired functionalities. Below is a detailed explanation of why I chose the DHT11 sensor, ultrasonic sensor, LDR sensor, relay module, and NodeMCU.

1. DHT11 Sensor

Purpose: The DHT11 sensor is used to monitor the room temperature.

Reason for Choice: The DHT11 is a digital sensor providing reliable and accurate temperature and humidity readings. It is cost-effective, easy to interface with microcontrollers, and provides sufficient accuracy for home automation purposes. In this project, it helps to determine if the room temperature exceeds 60 degrees Celsius, a condition under which the fan should be turned on if a person is present in the room.

2. Ultrasonic Sensor

Purpose: The ultrasonic sensor is used to detect the presence of a person in the room.

Reason for Choice: Ultrasonic sensors are highly effective for presence detection due to their ability to measure distance by sending and receiving sound waves. They are reliable in various lighting conditions, unlike infrared sensors, making them suitable for detecting human presence in different environments. The sensor helps in ensuring that the fan and bulb operate only when someone is in the room, thereby saving energy.

3. LDR Sensor

Purpose: The LDR (Light Dependent Resistor) sensor is used to determine if it is day or night.

Reason for Choice: LDRs are simple and effective for detecting light intensity. They change resistance based on the amount of light falling on them, making them ideal for distinguishing between day and night conditions. This sensor ensures that the bulb only turns on when it is dark and a person is present in the room, further optimizing energy usage.

4. 2-Channel Relay Module

Purpose: The relay module is used to control the fan and bulb.

Reason for Choice: Relays are essential for controlling high voltage AC devices like fans and bulbs with a low voltage DC signal from the microcontroller. A 2-channel relay module allows us to independently control two devices (the fan and the bulb) with a single microcontroller. This setup provides the necessary isolation and control, ensuring safe operation of high voltage devices.

5. NodeMCU Board

Purpose: NodeMCU is the microcontroller unit used to integrate all sensors and control the relays.

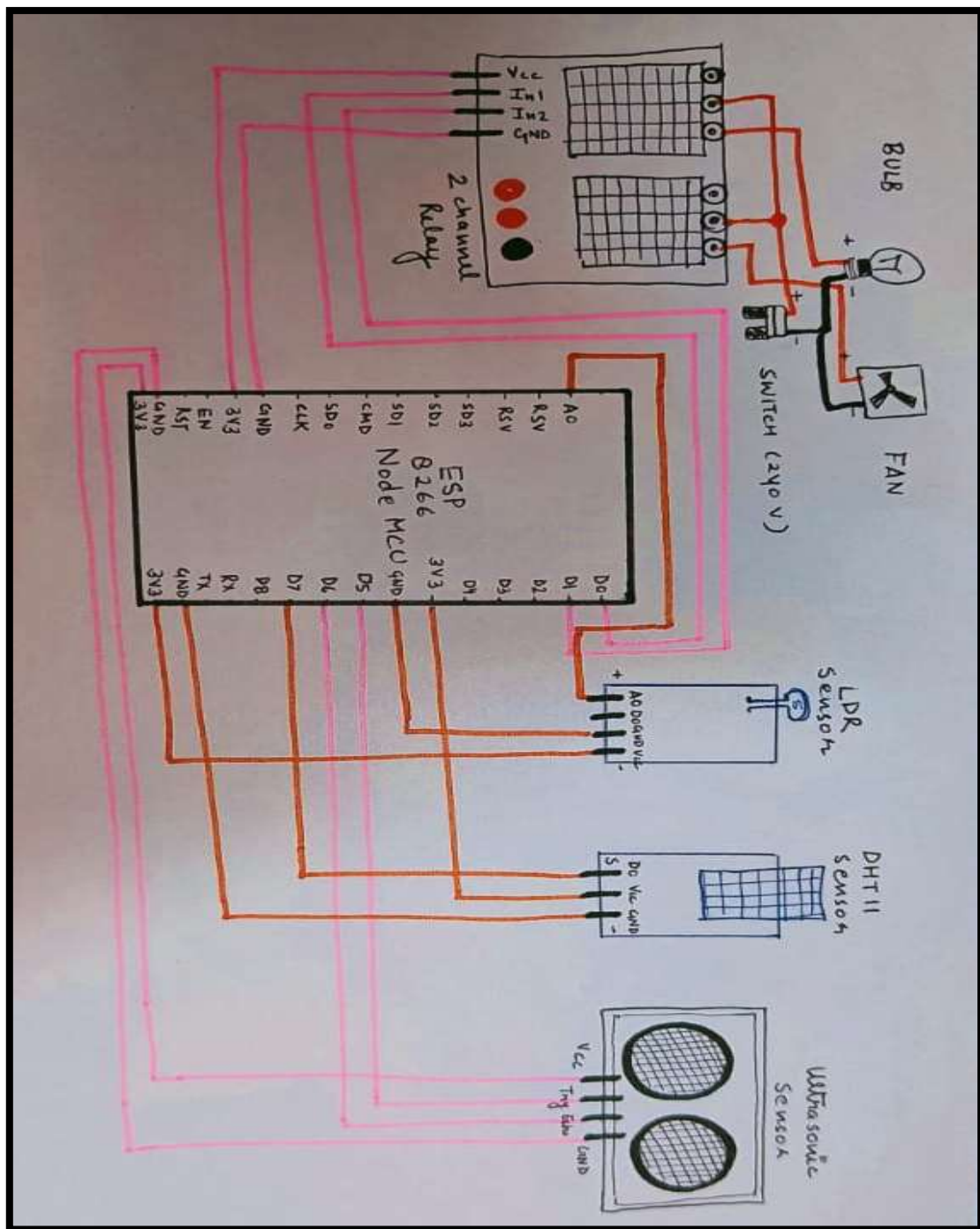
Reason for Choice: NodeMCU is a versatile, low-cost microcontroller with built-in WiFi capabilities, making it ideal for IoT and home automation projects. It supports the Arduino IDE, making it easy to program and deploy. The WiFi functionality allows for remote control and monitoring of the system via a mobile application. Additionally, NodeMCU has sufficient GPIO pins to connect all the sensors and relays, making it a compact and efficient choice for this project.

Below is the pin and connection diagram of the project:

List of Hardware Components Used:

Serial Number	Description of Hardware	Quantity	Remarks if any
1.	DHT11 Sensor	1	To measure temperature of room
2.	LDR Sensor	1	To detect if it's day or night
3.	Ultrasonic Sensor	1	To detect presence of person in the room
4.	NodeMCU	1	ESP8266 Wifi inbuilt development board

5.	2 Channel Relay	1	To connect the setup to AC voltage of 240 V
6.	Bulb	1	Home appliance(240V)
7.	Fan	1	Home appliance
8.	Jumper wires	40	M-M,M-F,F-M



Software Architecture

[A] Functional Blocks

1. Initialization Block:

- `setup()`
- Purpose: Initializes all components and sets up the initial state.
- Sub-functions: `serialSetup()`, `DHTsetup()`, `LDRsetup()`, `ultraSonicSensorSetup()`, `relaySetup()`, `wifiSetup()`

2. Main Control Loop:

- `loop()`
- Purpose: Continuously checks the system state and manages automatic and manual modes.
- Sub-functions: `handleAutoMode()`, `handleManualMode()`, `switchMode()`

3. Sensor Handling Block:

- `DHTsetup()`, `getTemp()`, `LDRsetup()`, `isItLight()`, `ultraSonicSensorSetup()`, `getDistance()`
- Purpose: Initializes and reads data from sensors.

4. Relay Control Block:

- `relaySetup()`, `setRelay()`
- Purpose: Controls the relays to turn on/off the fan and bulb.

5. WiFi and Server Block:

- `wifiSetup()`
- Purpose: Manages WiFi connection and sets up the server for handling HTTP requests.

6. Blynk Integration Block:

- Blynk button handlers (`BLYNK_WRITE(V0)`, `BLYNK_WRITE(V1)`)
- Purpose: Handles manual control inputs from the Blynk app.

[B] Functionality of Each Block

1. Initialization Block

- **serialSetup():** Initializes serial communication.

```
void serialSetup() {  
    Serial.begin(115200);  
    Serial.println("");  
}
```

Pseudocode:

Start Serial communication at 115200 baud rate
Print an empty line to the serial monitor

- **DHTsetup():** Initializes the DHT sensor.

```
void DHTsetup() {  
    Serial.println("DHTxx test!");  
    dht.begin();  
}
```

Pseudocode:

Print "DHTxx test!" to serial monitor
Initialize the DHT sensor

- **LDRsetup():** Sets up the LDR sensor.

```
void LDRsetup() {  
    pinMode(LDR_PIN, INPUT);  
}
```

Pseudocode:

Set LDR_PIN as INPUT

- **ultraSonicSensorSetup():** Initializes the ultrasonic sensor.

```
void ultraSonicSensorSetup() {  
    pinMode(TRIG_PIN, OUTPUT);  
    pinMode(ECHO_PIN, INPUT);  
}
```

Pseudocode:

Set TRIG_PIN as OUTPUT

Set ECHO_PIN as INPUT

- **relaySetup():** Sets up the relay pins.

```
void relaySetup() {  
  pinMode(RELAY_PIN1, OUTPUT);  
  pinMode(RELAY_PIN2, OUTPUT);  
  digitalWrite(RELAY_PIN1, HIGH);  
  digitalWrite(RELAY_PIN2, HIGH);  
}
```

Pseudocode:

Set RELAY_PIN1 as OUTPUT

Set RELAY_PIN2 as OUTPUT

Set both relay pins to HIGH (turn off relays)

- **wifiSetup():** Connects to WiFi and initializes the server and Blynk.

```
void wifiSetup() {  
  Serial.print("Connecting to ");  
  Serial.println(wifi_name);  
  WiFi.begin(wifi_name, wifi_pass);  
  while (WiFi.status() != WL_CONNECTED) {  
    delay(1000);  
    Serial.print(".");  
  }  
  Serial.println("");  
  Serial.println("WiFi connected.");  
  server.begin();  
  Blynk.begin(auth, wifi_name, wifi_pass, "blynk.cloud", 80);  
}
```

Pseudocode:

Print "Connecting to " and WiFi SSID to the serial monitor

Connect to WiFi using provided SSID and password

While not connected, print "." every second

Print "WiFi connected" to serial monitor

Start WiFi server

2. Main Control Loop

- **loop():** Main control loop, switches between manual and auto modes.

```
void loop() {  
  if (WiFi.status() != WL_CONNECTED) {  
    wifiSetup();  
  }  
  
  WiFiClient client = server.available();  
  if (client) {  
    String request = client.readStringUntil('\n');  
    client.flush();  
  
    if (request.indexOf("/manual/on") != -1) {  
      switchMode(true);  
    }  
    if (request.indexOf("/manual/off") != -1) {  
      switchMode(false);  
    }  
  
    client.println("HTTP/1.1 200 OK");  
    client.println("Content-Type: text/html");  
    client.println("");  
    client.println("<DOCTYPE HTML>");  
    client.println("<html>");  
    client.println("<h1>Home Automation System</h1>");  
    client.println("<p>Manual Mode: " + String(isManualMode ? "ON" : "OFF") + "</p>");  
    client.println("<p><a href='/manual/on/'>Turn Manual Mode ON</a></p>");  
    client.println("<p><a href='/manual/off/'>Turn Manual Mode OFF</a></p>");  
    client.println("</html>");  
    client.stop();  
  }  
  
  if (isManualMode) {  
    handleManualMode();  
  } else {  
    handleAutoMode();  
  }  
}
```

Pseudocode:

Setup:

- Call serialSetup()
- Call DHTsetup()
- Call LDRsetup()
- Call ultraSonicSensorSetup()
- Call relaySetup()
- Call wifiSetup()

Loop:

- Call handleManualMode()
- If isManualMode is false, call handleAutoMode()

- **handleAutoMode():** Manages the automatic control of the fan and bulb.

```
void handleAutoMode() {
  if (!isManualMode) {
    float distance = getDistance();
    float temp = getTemp();
    bool isLight = isitLight();

    if (distance <= 30) {
      if (temp >= 60) {
        setRelay(RELAY_PIN2, LOW); // Turn fan ON
      } else {
        setRelay(RELAY_PIN2, HIGH); // Turn fan OFF
      }
      if (isLight) {
        setRelay(RELAY_PIN1, HIGH); // Turn light OFF
      } else {
        setRelay(RELAY_PIN1, LOW); // Turn light ON
      }
    } else {
      setRelay(RELAY_PIN1, HIGH); // Turn light OFF
      setRelay(RELAY_PIN2, HIGH); // Turn fan OFF
    }
  }
}
```

Pseudocode:

Get distance from ultrasonic sensor

Get temperature from DHT sensor

Get light status from LDR sensor

If distance is less than or equal to 30 cm:

 If temperature is 60 degrees or more:

 Turn on fan relay

 Else:

 Turn off fan relay

 If it is dark:

 Turn on light relay

 Else:

 Turn off light relay

Else:

 Turn off both relays

- **handleManualMode():** Handles manual control via the Blynk app.

```
void handleManualMode() {
  Blynk.run();
}
```

Pseudocode:

If client is available:

 Read request from client

 If request contains "/manual/on", set isManualMode to true

 If request contains "/manual/off", set isManualMode to false

 If request contains "/light/on" and isManualMode is true, turn on light relay

 If request contains "/light/off" and isManualMode is true, turn off light relay

 If request contains "/fan/on" and isManualMode is true, turn on fan relay

 If request contains "/fan/off" and isManualMode is true, turn off fan relay

 Send response to client

- **switchMode(bool manual):** Switches between manual and automatic modes.

```
void switchMode(bool manual) {  
    isManualMode = manual;  
    if (isManualMode) {  
        setRelay(RELAY_PIN1, HIGH); // Turn off light in manual mode initially  
        setRelay(RELAY_PIN2, HIGH); // Turn off fan in manual mode initially  
    }  
}
```

3. Sensor Handling Block

- **getTemp():** Reads temperature from the DHT sensor.

```
float getTemp() {  
    delay(2000);  
    float h = dht.readHumidity();  
    float t = dht.readTemperature();  
    if (isnan(h) || isnan(t)) {  
        Serial.println("Failed to read from DHT sensor");  
        return 0;  
    }  
    Serial.print("Temperature: ");  
    Serial.print(t);  
    Serial.println(" C");  
    return t;  
}
```

Pseudocode:

Delay for 2 seconds

Read humidity and temperature from DHT sensor

If readings are NaN, print error message and return 0
Print temperature to serial monitor
Return temperature

- **isitLight():** Reads the state of the LDR sensor.

```
bool isitLight() {  
    int lightState = digitalRead(LDR_PIN);  
    if (lightState == HIGH) {  
        Serial.println("It is dark!");  
    } else {  
        Serial.println("It is light!");  
    }  
    return lightState;  
}
```

Pseudocode:

Read the state of LDR_PIN
If state is HIGH, print "It is dark!" to serial monitor
Else, print "It is light!" to serial monitor
Return the state

- **getDistance():** Reads the distance from the ultrasonic sensor.

```
float getDistance() {  
    digitalWrite(TRIG_PIN, LOW);  
    delayMicroseconds(2);  
    digitalWrite(TRIG_PIN, HIGH);  
    delayMicroseconds(10);  
    digitalWrite(TRIG_PIN, LOW);  
    long duration = pulseIn(ECHO_PIN, HIGH);  
    float distanceCm = duration * 0.034 / 2;  
    Serial.print("Distance in CM: ");  
    Serial.println(distanceCm);  
    delay(1000);  
    return distanceCm;  
}
```

Pseudocode:

Set TRIG_PIN to LOW for 2 microseconds
Set TRIG_PIN to HIGH for 10 microseconds
Set TRIG_PIN to LOW
Measure pulse duration from ECHO_PIN
Calculate distance in cm
Print distance to serial monitor
Delay for 1 second
Return distance

4. Relay Control Block

- **setRelay(uint8_t relay_num, uint8_t val):** Controls the relay pins to switch the fan and bulb on/off.

```
void setRelay(uint8_t relay_num, uint8_t val) {  
    if (relay_num == RELAY_PIN1) {  
        Serial.print("Light relay: ");  
        Serial.println(val);  
        digitalWrite(RELAY_PIN1, val);  
    } else if (relay_num == RELAY_PIN2) {  
        Serial.print("Fan relay: ");  
        Serial.println(val);  
        digitalWrite(RELAY_PIN2, val);  
    }  
}
```

Pseudocode:

If relay_num is RELAY_PIN1, print "Light relay: " and val to serial monitor, set RELAY_PIN1 to val

If relay_num is RELAY_PIN2, print "Fan relay: " and val to serial monitor, set RELAY_PIN2 to val

[C] Ensuring Requirements are Met

- **Light Bulb Activation:** The bulb activates within one minute of detecting a human presence and darkness.
- **Fan Activation:** The fan activates within one minute of the room temperature exceeding the threshold.
- **Manual Control Mode:** The bulb and fan can be manually controlled via the Blynk app when in manual mode.
- **Automatic Mode Priority:** If the system is in automatic mode, manual control is disabled.
- **Environmental Requirements:** The system operates effectively within the specified temperature range.

[D] Pseudo Code

function setup:

call serialSetup

call DHTsetup

call LDRsetup

call ultraSonicSensorSetup

call relaySetup

call wifiSetup

function loop:

if WiFi is not connected:

call wifiSetup

if client is available:

read and process client request

if request is to turn on manual mode:

call switchMode(true)

if request is to turn off manual mode:

call switchMode(false)

respond to client with current mode status

if isManualMode is true:

call handleManualMode

Code

```
1  #define BLYNK_TEMPLATE_ID "TMPL34TQXUmqO"
2  #define BLYNK_TEMPLATE_NAME "HomeAutomation"
3  #define BLYNK_PRINT Serial
4
5  #include <ESP8266WiFi.h>
6  #include <BlynkSimpleEsp8266.h>
7  #include "DHT.h"
8
9  // Pin Definitions
10 const int DHTPIN = D7;
11 #define DHTTYPE DHT11
12 const int LDR_PIN = A0;
13 const int TRIG_PIN = D5;
14 const int ECHO_PIN = D6;
15 const int RELAY_PIN1 = D0;
16 const int RELAY_PIN2 = D1;
17
18 // WiFi Credentials
19 const char* wifi_name = "realme 8 5G"; // Enter your WIFI name
20 const char* wifi_pass = "87654321"; // Enter your WIFI password
21
22 // Blynk Credentials
23 #define BLYNK_AUTH_TOKEN "sYftbs1siF4g0sMpFCJXESijPbvppTr5"
24 char auth[] = BLYNK_AUTH_TOKEN;
25
26 // Create instances
27 DHT dht(DHTPIN, DHTTYPE);
28 WiFiServer server(80);
29
30 // Global Variables
31 bool isManualMode = false;
32
33 // Function Prototypes
34 void serialSetup();
35 void DHTsetup();
36 float getTemp();
37 void LDRsetup();
38 bool isitLight();
39 void ultraSonicSensorSetup();
40 float getDistance();
41 void relaySetup();
42 void setRelay(uint8_t relay_num, uint8_t val);
43 void wifiSetup();
44 void handleAutoMode();
45 void handleManualMode();
46 void switchMode(bool manual);
47
```

```

48 // Blynk button handlers
49 BLYNK_WRITE(V0) {
50   if (isManualMode) {
51     digitalWrite(RELAY_PIN1, param.asInt());
52   }
53 }
54
55 BLYNK_WRITE(V1) {
56   if (isManualMode) {
57     digitalWrite(RELAY_PIN2, param.asInt());
58   }
59 }
60
61 void serialSetup() {
62   Serial.begin(115200);
63   Serial.println("");
64 }
65
66 void DHTsetup() {
67   Serial.println("DHTxx test!");
68   dht.begin();
69 }
70
71 float getTemp() {
72   delay(2000);
73   float h = dht.readHumidity();
74   float t = dht.readTemperature();
75   if (isnan(h) || isnan(t)) {
76     Serial.println("Failed to read from DHT sensor");
77     return 0;
78   }
79   Serial.print("Temperature: ");
80   Serial.print(t);
81   Serial.println(" C");
82   return t;
83 }
84
85 void LDRsetup() {
86   pinMode(LDR_PIN, INPUT);
87 }
88
89 bool isItLight() {
90   int lightState = digitalRead(LDR_PIN);
91   if (lightState == HIGH) {
92     Serial.println("It is dark!");
93   } else {
94     Serial.println("It is light!");
95   }

```

```

96     return lightState;
97 }
98
99 void ultraSonicSensorSetup() {
100     pinMode(TRIG_PIN, OUTPUT);
101     pinMode(ECHO_PIN, INPUT);
102 }
103
104 float getDistance() {
105     digitalWrite(TRIG_PIN, LOW);
106     delayMicroseconds(2);
107     digitalWrite(TRIG_PIN, HIGH);
108     delayMicroseconds(10);
109     digitalWrite(TRIG_PIN, LOW);
110     long duration = pulseIn(ECHO_PIN, HIGH);
111     float distanceCm = duration * 0.034 / 2;
112     Serial.print("Distance in CM: ");
113     Serial.println(distanceCm);
114     delay(1000);
115     return distanceCm;
116 }
117
118 void relaySetup() {
119     pinMode(RELAY_PIN1, OUTPUT);
120     pinMode(RELAY_PIN2, OUTPUT);
121     digitalWrite(RELAY_PIN1, HIGH);
122     digitalWrite(RELAY_PIN2, HIGH);
123 }
124
125 void setRelay(uint8_t relay_num, uint8_t val) {
126     if (relay_num == RELAY_PIN1) {
127         Serial.print("Light relay: ");
128         Serial.println(val);
129         digitalWrite(RELAY_PIN1, val);
130     } else if (relay_num == RELAY_PIN2) {
131         Serial.print("Fan relay: ");
132         Serial.println(val);
133         digitalWrite(RELAY_PIN2, val);
134     }
135 }
136
137 void wifiSetup() {
138     Serial.print("Connecting to ");
139     Serial.println(wifi_name);
140     WiFi.begin(wifi_name, wifi_pass);
141     while (WiFi.status() != WL_CONNECTED) {
142         delay(1000);

```

```

143     Serial.print(".");
144 }
145 Serial.println("");
146 Serial.println("WiFi connected.");
147 server.begin();
148 Blynk.begin(auth, wifi_name, wifi_pass, "blynk.cloud", 80);
149 }
150
151 void handleAutoMode() {
152     if (!isManualMode) {
153         float distance = getDistance();
154         float temp = getTemp();
155         bool isLight = isitLight();
156
157         if (distance <= 30) {
158             if (temp >= 60) {
159                 setRelay(RELAY_PIN2, LOW); // Turn fan ON
160             } else {
161                 setRelay(RELAY_PIN2, HIGH); // Turn fan OFF
162             }
163             if (isLight) {
164                 setRelay(RELAY_PIN1, HIGH); // Turn light OFF
165             } else {
166                 setRelay(RELAY_PIN1, LOW); // Turn light ON
167             }
168         } else {
169             setRelay(RELAY_PIN1, HIGH); // Turn light OFF
170             setRelay(RELAY_PIN2, HIGH); // Turn fan OFF
171         }
172     }
173 }
174
175 void handleManualMode() {
176     Blynk.run();
177 }
178
179 void switchMode(bool manual) {
180     isManualMode = manual;
181     if (isManualMode) {
182         setRelay(RELAY_PIN1, HIGH); // Turn off light in manual mode initially
183         setRelay(RELAY_PIN2, HIGH); // Turn off fan in manual mode initially
184     }
185 }
186
187 void setup() {
188     serialSetup();
189     DHTsetup();
190     LDRsetup();
191     ultraSonicSensorSetup();
192     relaySetup();

```

```

193     wifiSetup();
194 }
195
196 void loop() {
197     if (WiFi.status() != WL_CONNECTED) {
198         wifiSetup();
199     }
200
201     WiFiClient client = server.available();
202     if (client) {
203         String request = client.readStringUntil('\r');
204         client.flush();
205
206         if (request.indexOf("/manual/on") != -1) {
207             switchMode(true);
208         }
209         if (request.indexOf("/manual/off") != -1) {
210             switchMode(false);
211         }
212
213         client.println("HTTP/1.1 200 OK");
214         client.println("Content-Type: text/html");
215         client.println("");
216         client.println("<!DOCTYPE HTML>");
217
218         client.println("<html>");
219         client.println("<h1>Home Automation System</h1>");
220         client.println("<p>Manual Mode: " + String(isManualMode ? "ON" : "OFF") + "</p>");
221         client.println("<p><a href=\"/manual/on\">Turn Manual Mode ON</a></p>");
222         client.println("<p><a href=\"/manual/off\">Turn Manual Mode OFF</a></p>");
223         client.println("</html>");
224         client.stop();
225     }
226
227     if (isManualMode) {
228         handleManualMode();
229     } else {
230         handleAutoMode();
231     }
232 }

```

Testing And Result

To ensure the functionality, reliability, and safety of the home automation system, I followed a comprehensive testing strategy. This involved unit testing, integration testing, and system testing, with each test designed to validate that the system meets the specified requirements and performs correctly under various conditions.

1. Unit Testing

Unit testing involved testing individual components or functions to ensure they worked as expected in isolation.

Sensor Testing:

DHT11 Sensor: I validated that the sensor correctly reads temperature and humidity values by mocking different temperature and humidity values and verifying the system's response. I checked if the temperature reading triggered the fan at the correct threshold.

Ultrasonic Sensor: I validated that the sensor correctly detects the presence of a person by simulating the presence and absence of a person and verifying if the detection logic worked correctly.

LDR Sensor: I validated that the sensor correctly detects day and night conditions by simulating different lighting conditions and checking if the system correctly identified them.

Relay Testing:

I ensured that the relays correctly switched the fan and bulb on and off based on control signals. I tested relays with manual and automatic control to verify proper switching.

WiFi and Blynk Integration Testing:

I verified that the system connected to the WiFi network and communicated with the Blynk server. I tested manual control through the Blynk app to ensure commands were received and executed correctly.

2. Integration Testing

Integration testing involved combining individual components and testing them as a group to ensure they worked together correctly.

Sensor and Relay Integration:

I verified that the sensors' readings correctly triggered the relays. I tested scenarios where the temperature exceeded the threshold and the fan should turn on, and scenarios where the presence was detected in a dark room and the bulb should turn on.

Manual and Automatic Mode Integration:

I tested switching between manual and automatic modes. I verified that the system disabled manual control when in automatic mode and vice versa. I ensured the system correctly prioritized automatic mode when activated.

3. System Testing

System testing involved testing the complete integrated system in a real-world environment to ensure it met the overall requirements.

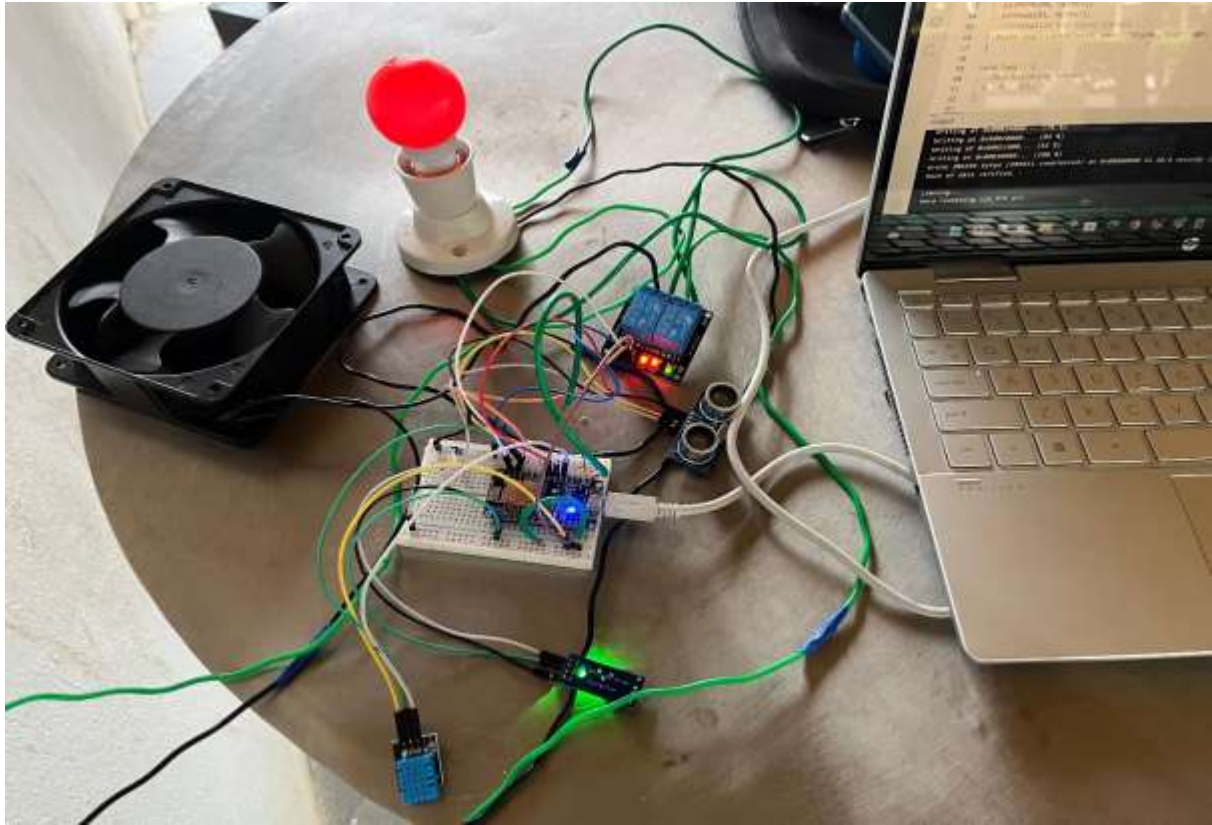
End-to-End Testing:

I tested the complete workflow from sensing conditions to actuating the fan and bulb. I verified that the system responded within the specified time (e.g., light bulb activation within one minute).

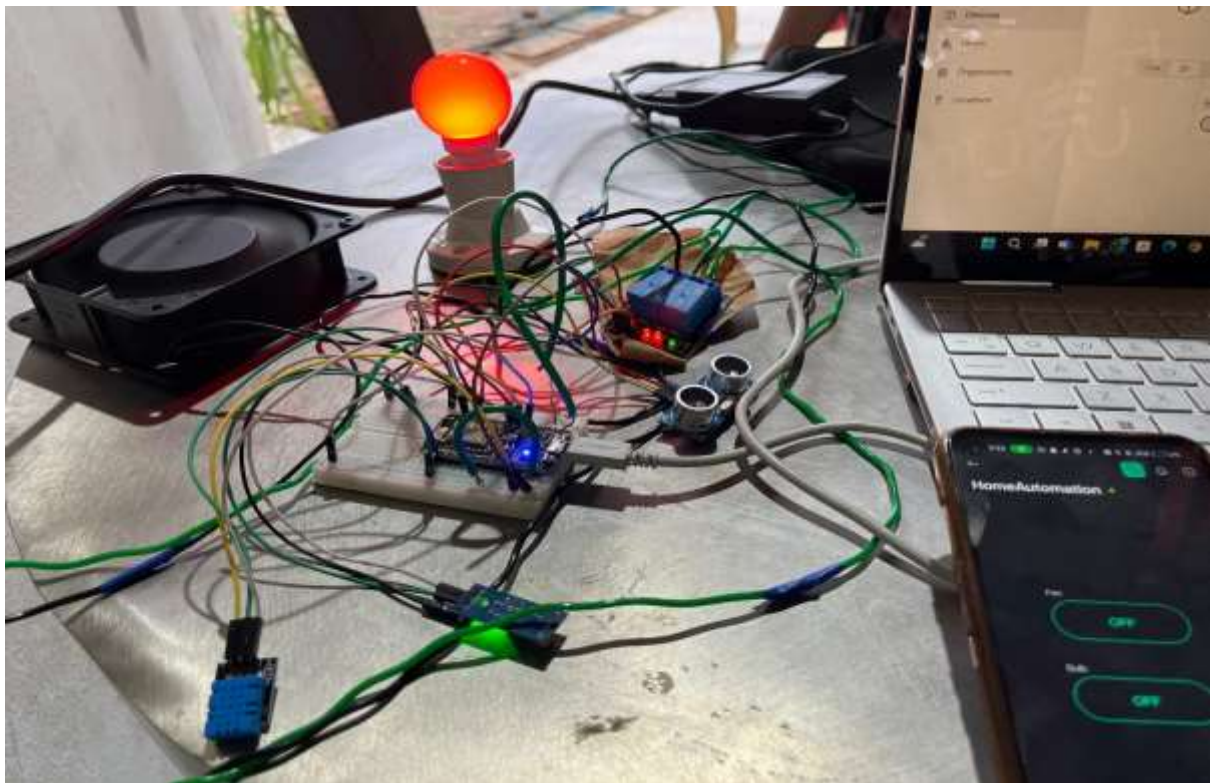
Performance Testing:

I tested the system's response time and ensured it met the performance requirements. I verified that the system operated correctly under different environmental conditions (temperature range of 0 to 60 degrees Celsius).

To verify the results, I recorded all test cases, expected outcomes, and actual results. I documented any deviations and the steps taken to address them. By following a structured testing and verification process, I ensured that the home automation system functioned correctly, met all specified requirements.



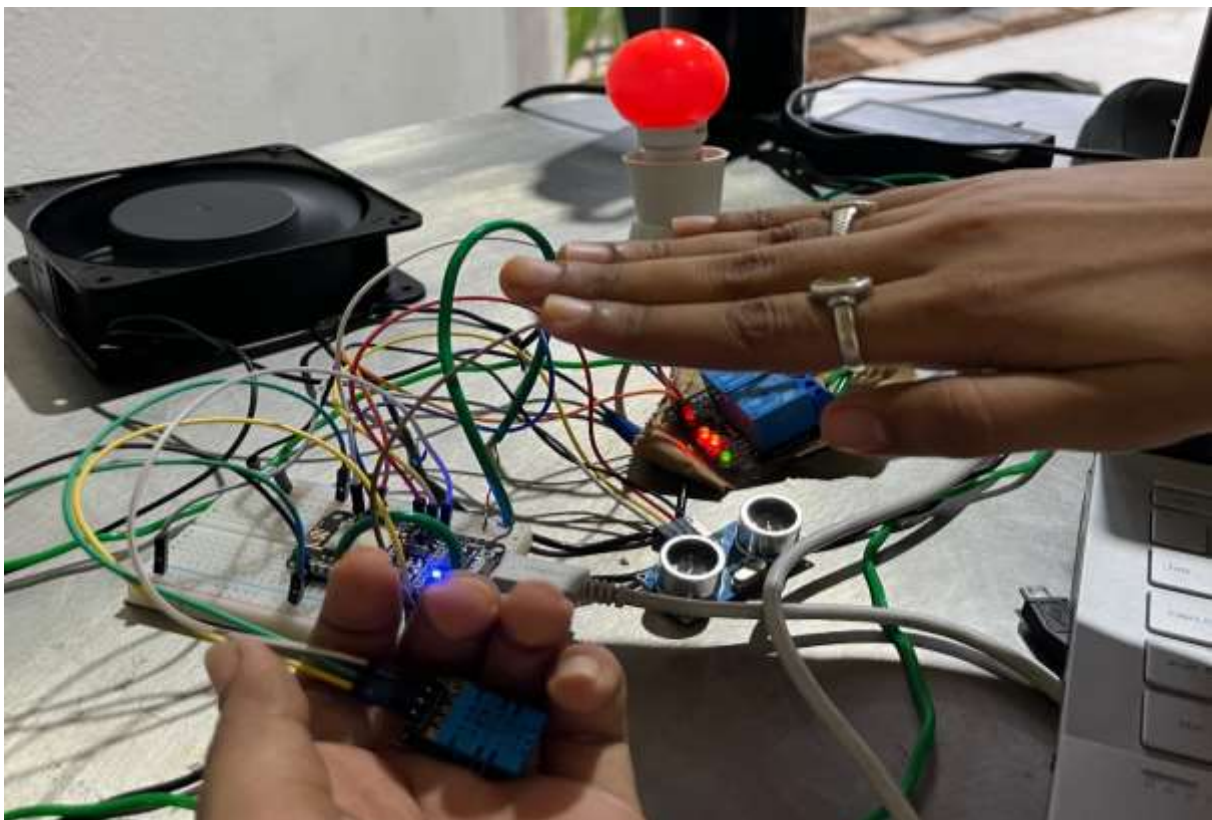
a. Home Automation Setup with LDR sensor,DHT11 sensor and ultrasonic sensor together with 2 channel relay, ESP8266 Node MCU, Fan and Bulb



b. Manual Mode using blynk.io application and Blynk.iot mobile app



- c. Automated mode: when my hands are on the ultrasonic sensor and on the LDR sensor. This shows the first condition of automated mode.



- d. Automated Mode: Fan running the sensor detects heat and person using DHT11 and ultrasonic sensors respectively. This shows the second condition of automated mode

Future Works

Keeping the goal to enhance the system's functionality, user experience, and adaptability to various environments. Here are some potential future work areas we can work on:

1. Integration with Smart Home Ecosystems:

Voice Control: Integrate the system with popular voice assistants like Amazon Alexa, Google Assistant, and Apple Siri. This will allow users to control the fan and bulb using voice commands, enhancing user convenience.

Smart Home Hubs: Connect the system with smart home hubs such as SmartThings or HomeKit to allow for more sophisticated automation rules and integration with other smart devices (e.g., smart thermostats, security cameras).

2. Advanced Sensor Integration:

CO2 and Air Quality Sensors: Adding sensors to monitor CO2 levels and air quality can help in maintaining a healthy indoor environment. The system could activate ventilation when poor air quality is detected.

Occupancy Sensors: Using more advanced occupancy sensors (e.g., infrared or microwave sensors) can improve the accuracy of detecting human presence, even when occupants are stationary.

3. Energy Efficiency Improvements:

Smart Energy Management: Implement algorithms to optimize energy usage based on patterns of occupancy and environmental conditions. For instance, the system can reduce fan speed or dim the lights when full power is not necessary.

Renewable Energy Integration: Integrate the system with renewable energy sources like solar panels to manage energy usage more sustainably.

4. Enhanced User Interface and Experience:

Mobile App Enhancements: Improve the Blynk app interface with more intuitive controls, better status displays, and custom automation rules. Implement push notifications to alert users about significant events (e.g., temperature thresholds, security alerts).

Web Dashboard: Develop a web-based dashboard for users to control the system, view usage statistics, and set automation rules.

5. Security Enhancements:

User Authentication: Implement robust authentication mechanisms to ensure that only authorized users can control the system.

Data Encryption: Ensure all data transmitted between the sensors, relays, and the user's devices is encrypted to prevent unauthorized access.

6. Scalability and Adaptability:

Modular Design: Design the system to be easily scalable. Users should be able to add more devices (e.g., additional lights, appliances) without significant reconfiguration.

Customization Options: Allow users to customize the system's behavior based on their preferences, such as setting different temperature thresholds for the fan or different times for the light activation.

7. Machine Learning and Predictive Analytics:

Usage Patterns Analysis: Use machine learning algorithms to analyze usage patterns and predict user behavior. The system can preemptively adjust settings for comfort and efficiency.

Predictive Maintenance: Implement predictive maintenance features to monitor the health of the system components and notify users of any potential issues before they become critical.

8. Integration with Environmental and Weather Data:

Weather API Integration: Integrate with weather APIs to adjust the home environment based on external weather conditions. For example, if it's forecasted to be hot, the system can pre-cool the room.

Conclusion

In the home automation system project, I successfully integrated various sensors and actuators to create a responsive and intelligent environment. By using a DHT11 sensor, I effectively monitored the room temperature, ensuring that the fan activates when the temperature exceeds 60 degrees Celsius. The ultrasonic sensor provided accurate detection of human presence, while the LDR sensor allowed me to determine the lighting conditions, enabling the bulb to turn on when it's dark and someone is in the room. The Node MCU, with its Wi-Fi capabilities, facilitated remote control and monitoring through the Blynk IoT app, providing users with the convenience of managing their home appliances from anywhere.

The use of a 2-channel relay allowed me to control both the fan and the bulb, ensuring that the system could handle multiple devices simultaneously. I implemented a seamless integration of manual and automated modes, where manual control via the Blynk app overrides the automated system, enhancing user flexibility. My choice of components was driven by their reliability, cost-effectiveness, and ease of integration with the Node MCU platform, which offered robust connectivity and processing power.

As I look to the future, there are numerous avenues for further enhancement. Integrating voice control through smart assistants like Alexa or Google Assistant can make the system even more user-friendly. Incorporating additional sensors for air quality and occupancy can improve the system's environmental monitoring capabilities. Enhancing energy efficiency algorithms and adding predictive maintenance features will further optimize performance and longevity. Developing a more advanced user interface with customizable automation rules can cater to individual preferences and needs, making my home automation system an even more valuable addition to any smart home.